



Universidade Federal de Itajubá
IMC

UNIFEI

Otimização de Processos de Coleta em Centros de Distribuição

Rafael Lemes, Tiago Antônio, Enzo Andrade, Breno Hirakawa



INTRODUÇÃO

- Com a evolução do e-commerce, o volume de pedidos processados cresceu muito na última década.
- Essa escala pede soluções mais inteligentes de logística, que viabilizem um fluxo eficiente mesmo em cenários complexos.





INTRODUÇÃO

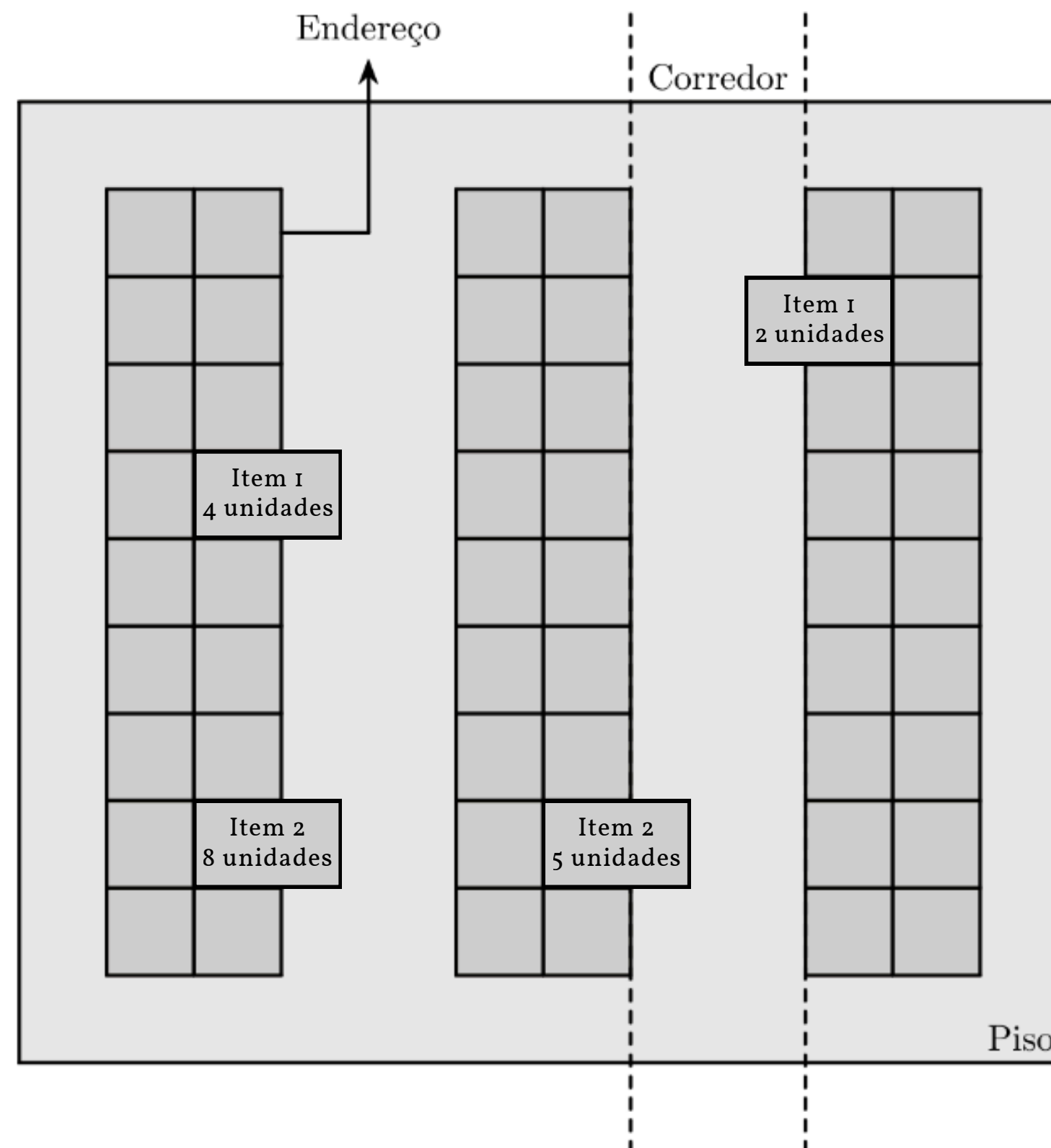


- Um dos problemas enfrentados por unidades de distribuição nesse contexto, é a **seleção ótima de pedidos**.
- O recolhimento de itens deve ocorrer de forma agrupada, minimizando retrabalho e custos operacionais.
- O problema é determinar de forma acertiva o melhor agrupamento de pedidos que otimiza o recolhimento.



DEFINIÇÃO DO PROBLEMA

- Para a definição desse problema, um centro de distribuição será classificado apenas em:
 - **Pedidos**
 - **Itens**
 - **Corredores**
- Cada **pedido** requer múltiplos itens em diferentes quantidades
- Cada **item** está em diferentes pedidos e em diferentes corredores
- Cada **corredor** pode conter diferentes itens e quantidades





DEFINIÇÃO DO PROBLEMA

- O conjunto de pedidos não processados é conhecido como **backlog**.
- Um conjunto de pedidos e um conjunto de corredores que validam uma operação de recolhimento é chamado de **wave**.
- O objetivo do problema é alcançar o maior valor objetivo em uma wave, ou seja: selecionar o conjunto de pedidos que maximize a quantidade de itens recolhidos minimizando os corredores visitados.

$$\text{Valor Objetivo} = \frac{\text{Total de itens recolhidos}}{\text{Total de corredores visitados}}$$



DEFINIÇÃO DO PROBLEMA

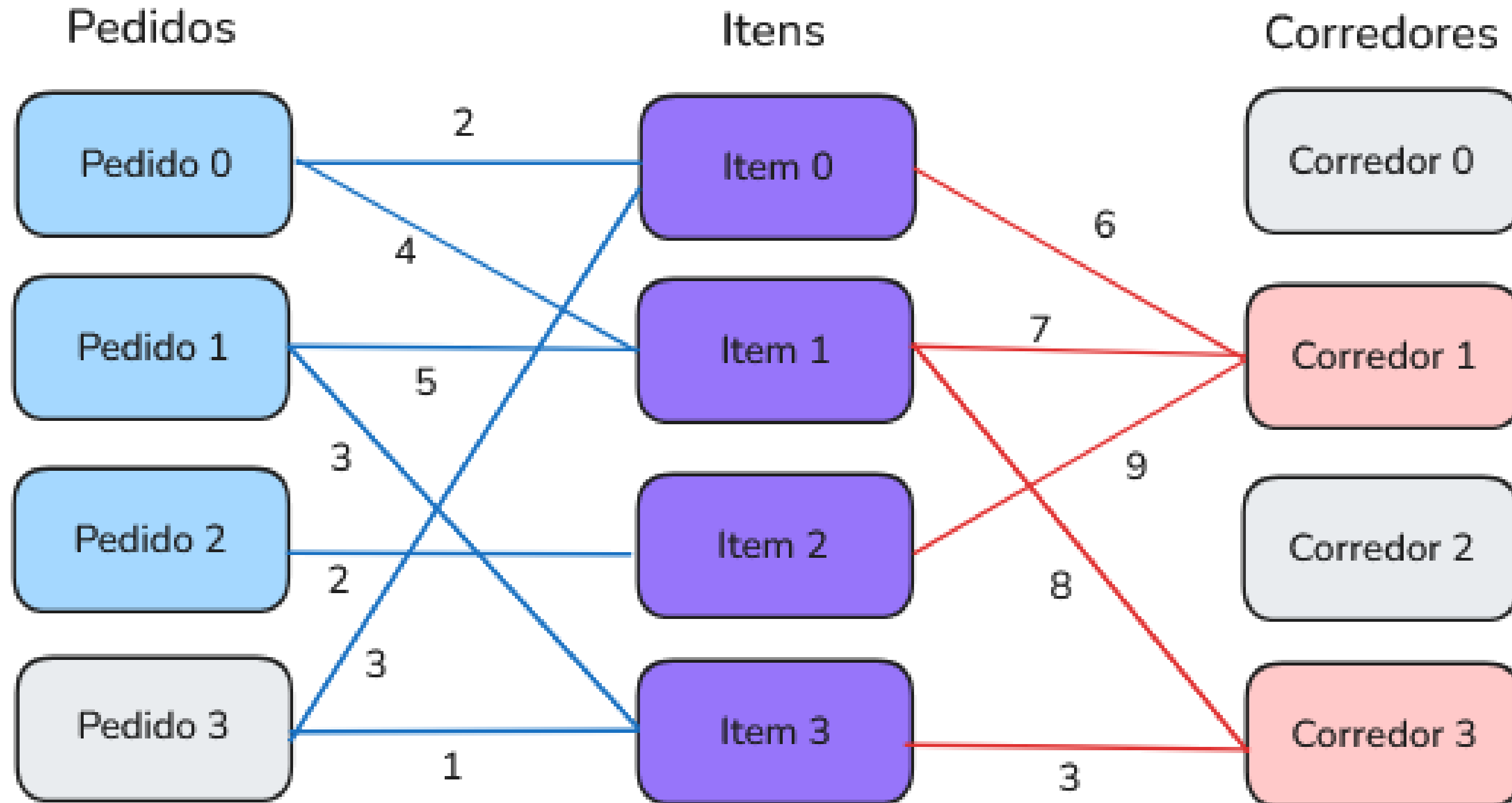
Porém, possuímos algumas restrições:

- Pedidos são recolhidos integralmente, e nenhum item é recolhido em avulso.
- Limite inferior da quantidade de itens em uma wave.
- Limite superior da quantidade de itens em uma wave.
- Os corredores possuem limitação de estoque, e essa barreira deve ser respeitada: uma wave válida não deve considerar mais itens do que estão disponíveis.



METODOLOGIA

Modelagem





METODOLOGIA

Dataset

```
61 155 116
```

```
3 13 1 36 1 135 1
```

```
4 20 1 86 1 97 1 123 1
```

```
4 2 2 21 1 84 1 142 1
```

...

```
30 68
```

} **Dimensão do cenário**

} **Composição dos Pedidos**

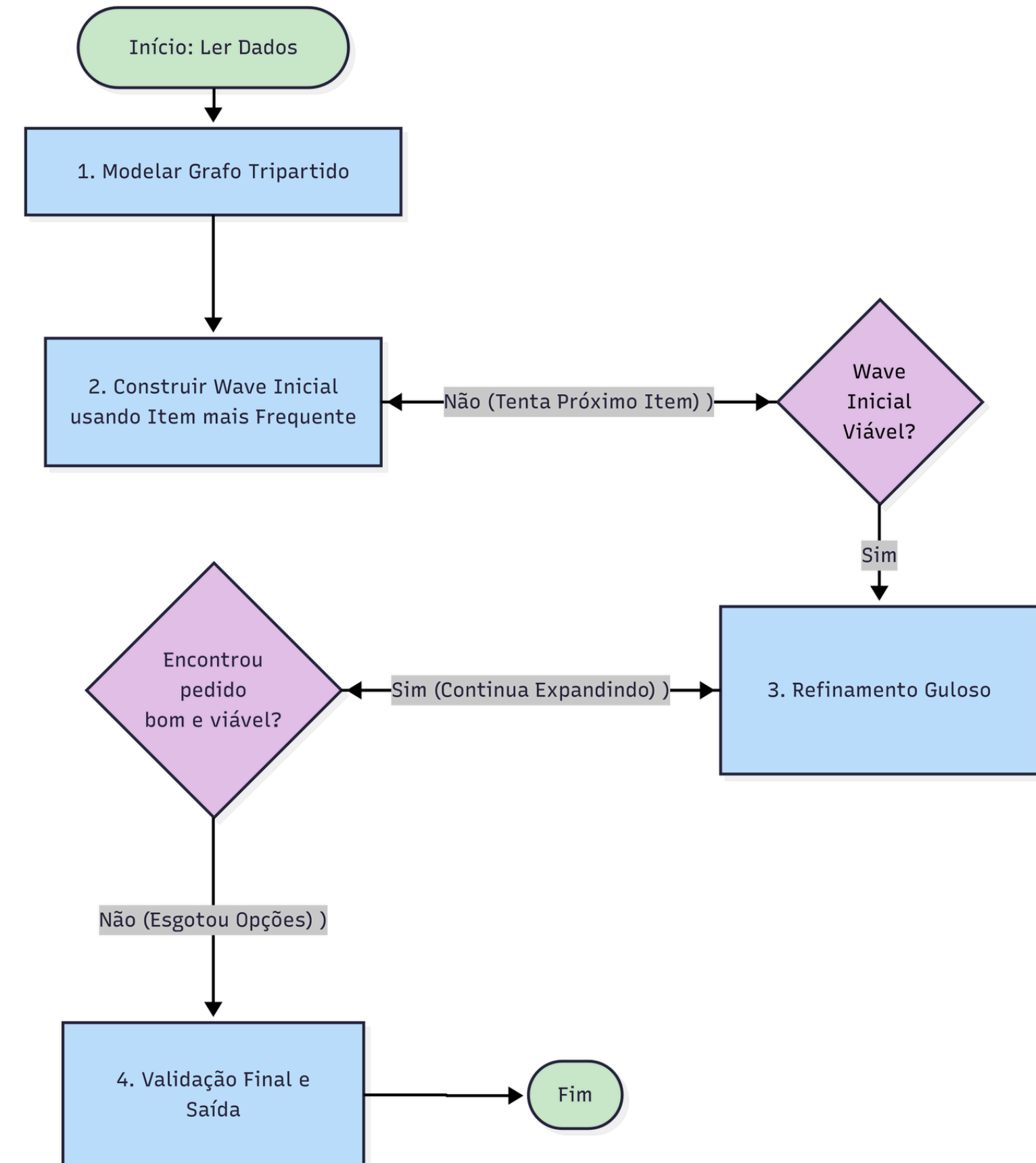
} **Limite inferior e superior**



METODOLOGIA

Desenvolvimento

- Algoritmo guloso de cobertura de conjuntos
- Estratégia de múltiplas sementes: são testados diferentes itens de alta demanda como ponto de partida

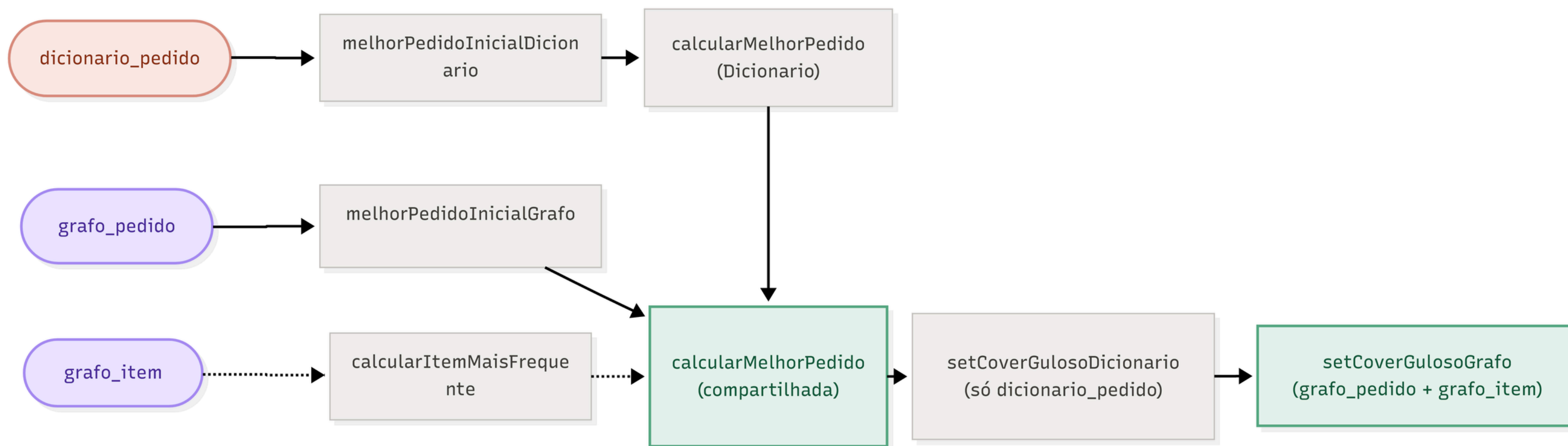




METODOLOGIA

Desenvolvimento

Abordagens Alternativas: Dicionários e listas, Grafos e dicionários





METODOLOGIA

Desenvolvimento

- Modelagem utilizando NetworkX
- Conexões entre pedidos e itens carregam o peso da demanda
- Conexões entre itens e corretores carregam o peso da disponibilidade de estoque

Algorithm 1: Modelagem do Grafo Tripartido Direcionado

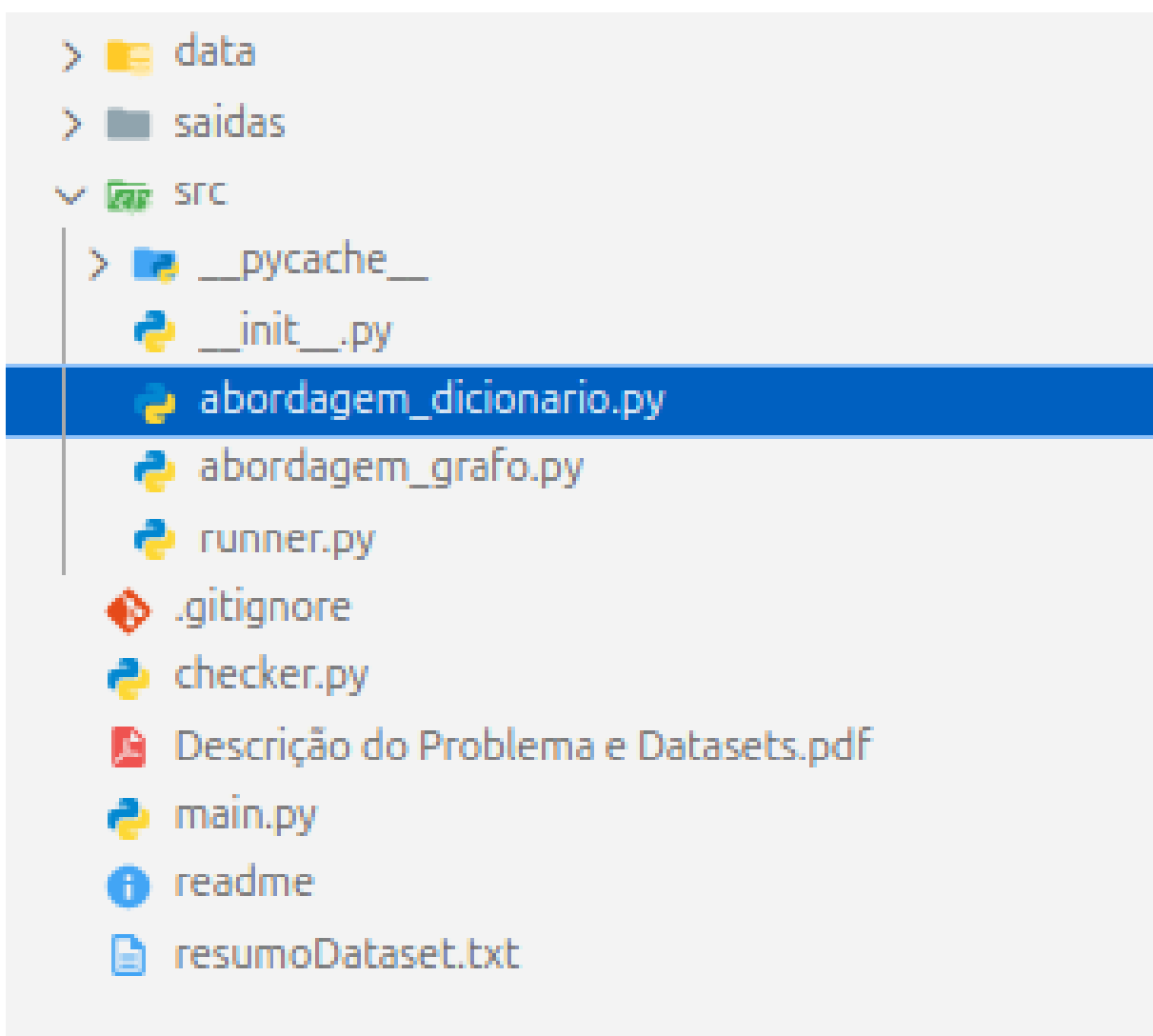
Input: Arquivo texto contendo limites da *wave*, demandas de pedidos e estoques dos corretores

Output: Grafo Tripartido *G*, Limites *waveMin* e *waveMax*

```
1 G ← Novo Grafo Direcionado vazio;
2 waveMin, waveMax ←
  LerLimitesOperacionais(Arquivo);
3 // Instanciação da camada de Pedidos e
  conexões de Demanda
4 for cada pedido ∈ Arquivo do
5   AdicionarNo(G, id = pedido, camada = 'pedido');
6   for cada (item, quantidade) ∈ pedido do
7     AdicionarNo(G, id = item, camada = 'item');
8     AdicionarAresta(G, origem = pedido, destino =
      item, peso = quantidade);
9   end
10 end
11 // Instanciação da camada de Corredores e
  conexões de Estoque
12 for cada corredor ∈ Arquivo do
13   AdicionarNo(G, id = corredor, camada =
    'corredor');
14   for cada (item, quantidade) ∈ corredor do
15     // O nó do item correspondente é
      referenciado na aresta
16     AdicionarAresta(G, origem = item, destino =
      corredor, peso = quantidade);
17   end
18 end
19 return G, waveMin, waveMax;
```



Codigo



```
#rediriciona para calculaPedidoInicial conforme abordagem utilizada
def calculaPedidoInicial(nomeAbordagem, pedidosValidos, corredores, grafo):
    if nomeAbordagem == 'dicionario_pedido':
        pedidoId = melhorPedidoInicialDicionario(pedidosValidos, corredores)
        melhoresPedidos = [pedidoId]

    elif nomeAbordagem == 'grafo_pedido':
        pedidoId = melhorPedidoInicialGrafo(grafo, pedidosValidos)
        melhoresPedidos = [pedidoId]

    elif nomeAbordagem == 'grafo_item':
        itemMaisFrequente = calcularItemMaisFrequente(grafo, pedidosValidos)
        melhoresPedidos = [
            pedidoId for pedidoId in pedidosValidos
            if itemMaisFrequente in pedidosValidos[pedidoId]
        ]

    else:
        raise ValueError(f"abordagem desconhecida: {nomeAbordagem}")

    quantidadePedidos = sum(
        sum(pedidosValidos[pedidoId].values()) for pedidoId in melhoresPedidos
    )
    return melhoresPedidos, quantidadePedidos

#rediriciona para calcularMelhorPedido conforme abordagem utilizada
```




Codigo

```
def organizaPedidosCorredores(filename):
    with open(filename, 'r') as f:
        linhas = f.readlines()

    primeiraLinha = list(map(int, linhas[0].split()))
    quantidadePedidos = primeiraLinha[0]
    quantidadeCorredor = primeiraLinha[2]

    pedidos = {}
    corredores = {}

    for i in range(quantidadePedidos):
        capturaItem = linhas[i + 1].split()
        numeros = []
        for j in capturaItem:
            numeros.append(int(j))
        items = {}
        k = 1
        while k < len(numeros):
            items[numeros[k]] = numeros[k + 1]
            k += 2
        pedidos[i] = items

    for i in range(quantidadeCorredor):
        capturaItem = linhas[1 + quantidadePedidos + i].split()
        numeros = []
        for j in capturaItem:
            numeros.append(int(j))
        items = {}
        k = 1
        while k < len(numeros):
            items[numeros[k]] = numeros[k + 1]
            k += 2
        corredores[i] = items

    ultimaLinha = linhas[-1].split()
    return pedidos, corredores, ultimaLinha[0], ultimaLinha[1]
```

```
: criaGrafo(pedidos, corredores):
    G = nx.DiGraph()
    for pedidoId, items in pedidos.items():
        G.add_node(('p', pedidoId), camada='pedido')
        for itemId, qtd in items.items():
            G.add_node(('i', itemId), camada='item')
            G.add_edge(('p', pedidoId), ('i', itemId), peso=qtd)
    for corredorId, items in corredores.items():
        G.add_node(('c', corredorId), camada='corredor')
        for itemId, qtd in items.items():
            G.add_node(('i', itemId), camada='item')
            G.add_edge(('i', itemId), ('c', corredorId), peso=qtd)
    return G
```



Codigo

```
def executaWave(pedidos, corredores, pedidosValidos, waveMin, waveMax,
               melhoresPedidos, quantidadePedidos, fnMelhorPedido, fnSetCover):

    demanda = calcularDemanda(pedidos, melhoresPedidos)
    melhoresCorredores = fnSetCover(demanda)

    if len(melhoresCorredores) == 0:
        return None, None, None

    wave = quantidadePedidos
    objetivoAtual = wave / len(melhoresCorredores)

    while wave < waveMax:
        proximoMelhorPedido = fnMelhorPedido(
            pedidosValidos, melhoresPedidos, melhoresCorredores, waveMax, wave
        )
        if proximoMelhorPedido is None:
            break
        novaWave = wave + sum(pedidosValidos[proximoMelhorPedido].values())
        if novaWave > waveMax:
            break
        novaDemanda = calcularDemanda(pedidosValidos, melhoresPedidos + [proximoMelhorPedido])
        if not verificaEstoqueDisponivel(novaDemanda, corredores):
            break
        novosCorredores = fnSetCover(novaDemanda)
        if len(novosCorredores) == 0:
            break
        novoObjetivo = novaWave / len(novosCorredores)
        if novoObjetivo >= objetivoAtual or wave < waveMin:
            melhoresPedidos.append(proximoMelhorPedido)
            wave = novaWave
            melhoresCorredores = novosCorredores
            objetivoAtual = novoObjetivo
        else:
            break
```

```
while wave < waveMin:
    adicionou = False
    for pedidoId in pedidosValidos:
        if pedidoId in melhoresPedidos:
            continue
        novaWave = wave + sum(pedidosValidos[pedidoId].values())
        if novaWave > waveMax:
            continue
        novaDemanda = calcularDemanda(pedidosValidos, melhoresPedidos + [pedidoId])
        if not verificaEstoqueDisponivel(novaDemanda, corredores):
            continue
        novosCorredores = fnSetCover(novaDemanda)
        if len(novosCorredores) == 0:
            continue
        melhoresPedidos.append(pedidoId)
        wave = novaWave
        melhoresCorredores = novosCorredores
        adicionou = True
        break
    if not adicionou:
        break

return melhoresPedidos, melhoresCorredores, objetivoAtual
```



Codigo

```
# Utiliza o setCover com o metodo guloso para gerar o melhor corredor
def setCoverGulosoDicionario(demanda, corredores):
    melhoresCorredores = []
    itemsRestantes = demanda.copy()
    estoqueAcumulado = {}
    while itemsRestantes:
        melhorCorredor = None
        melhorContribuicao = 0
        for corredorId, items in corredores.items():
            if corredorId in melhoresCorredores:
                continue
            contribuicao = 0
            for itemId, qtd in itemsRestantes.items():
                if itemId in items:
                    jaTemos = estoqueAcumulado.get(itemId, 0)
                    falta = qtd - jaTemos
                    if falta > 0:
                        contribuicao += min(items[itemId], falta)
            if contribuicao > melhorContribuicao:
                melhorContribuicao = contribuicao
                melhorCorredor = corredorId
        if melhorCorredor is None:
            break
        for itemId, qtd in corredores[melhorCorredor].items():
            if itemId in estoqueAcumulado:
                estoqueAcumulado[itemId] += qtd
            else:
                estoqueAcumulado[itemId] = qtd
        for itemId in list(itemsRestantes):
            if itemId in estoqueAcumulado:
                if estoqueAcumulado[itemId] >= itemsRestantes[itemId]:
                    itemsRestantes.pop(itemId)
        melhoresCorredores.append(melhorCorredor)
    return melhoresCorredores
```

```
def setCoverGulosoGrato(grato, demanda):
    corredoresSelecionados = []
    itemsRestantes = demanda.copy()
    estoqueAcumulado = {}
    candidatos = {n[1] for n in grato.nodes if n[0] == 'c'}

    while itemsRestantes:
        melhorCorredor = None
        melhorContribuicao = 0
        for corredorId in candidatos:
            if corredorId in corredoresSelecionados:
                continue
            contribuicao = 0
            noCorredor = ('c', corredorId)
            for noItem in grato.predecessors(noCorredor):
                itemId = noItem[1]
                if itemId in itemsRestantes:
                    disponivel = grato[noItem][noCorredor]['peso']
                    falta = itemsRestantes[itemId] - estoqueAcumulado.get(itemId, 0)
                    if falta > 0:
                        contribuicao += min(disponivel, falta)
            if contribuicao > melhorContribuicao:
                melhorContribuicao = contribuicao
                melhorCorredor = corredorId

        if melhorCorredor is None:
            break
        noCorredor = ('c', melhorCorredor)
        for noItem in grato.predecessors(noCorredor):
            itemId = noItem[1]
            qtd = grato[noItem][noCorredor]['peso']
            estoqueAcumulado[itemId] = estoqueAcumulado.get(itemId, 0) + qtd
        for itemId in list(itemsRestantes):
            if estoqueAcumulado.get(itemId, 0) >= itemsRestantes[itemId]:
                itemsRestantes.pop(itemId)
        corredoresSelecionados.append(melhorCorredor)
    return corredoresSelecionados
```




Codigo

```
#Gera a saida com os pedidos e corredores.
def gerarSaida(pedidosSelecionados, corredoresAtivos, filename="saida.txt"):
    with open(filename, 'w') as f:
        f.write(f"{len(pedidosSelecionados)}\n")
        for pedidoId in pedidosSelecionados:
            f.write(f"{pedidoId}\n")
        f.write(f"{len(corredoresAtivos)}\n")
        for corredorId in corredoresAtivos:
            f.write(f"{corredorId}\n")
```

Valores retornados do programa ao executar uma instância.

```
Instância 0004 [dicionario_pedido] | unidades=10 | corredores=6 | objetivo=1.67 | tempo=0.0011s
Instância 0004 [grafo_pedido] | unidades=19 | corredores=9 | objetivo=2.11 | tempo=0.0016s
Instância 0004 [grafo_item] | unidades=9 | corredores=6 | objetivo=1.50 | tempo=0.0006s
```

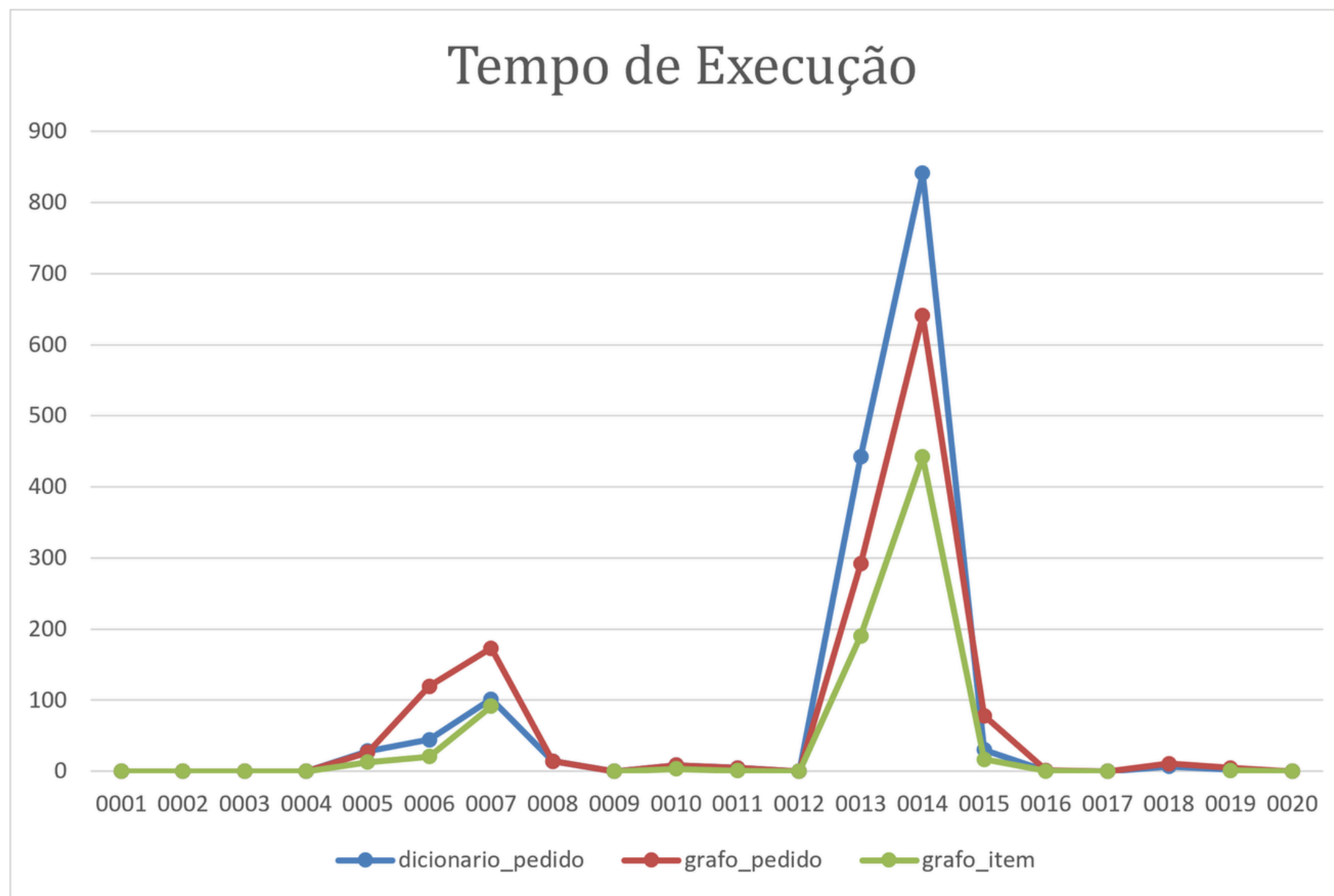
```
Instância 0007 [dicionario_pedido] | unidades=1353 | corredores=43 | objetivo=31.47 | tempo=101.5776s
Instância 0007 [grafo_pedido] | unidades=1316 | corredores=43 | objetivo=30.60 | tempo=173.1390s
Instância 0007 [grafo_item] | unidades=1381 | corredores=45 | objetivo=30.69 | tempo=91.5551s
```

saidas > saida_0004_dicionario_pedido.txt

```
1 4
2 7
3 1
4 4
5 5
6 6
7 17
8 19
9 54
10 83
11 12
12 80
13
```

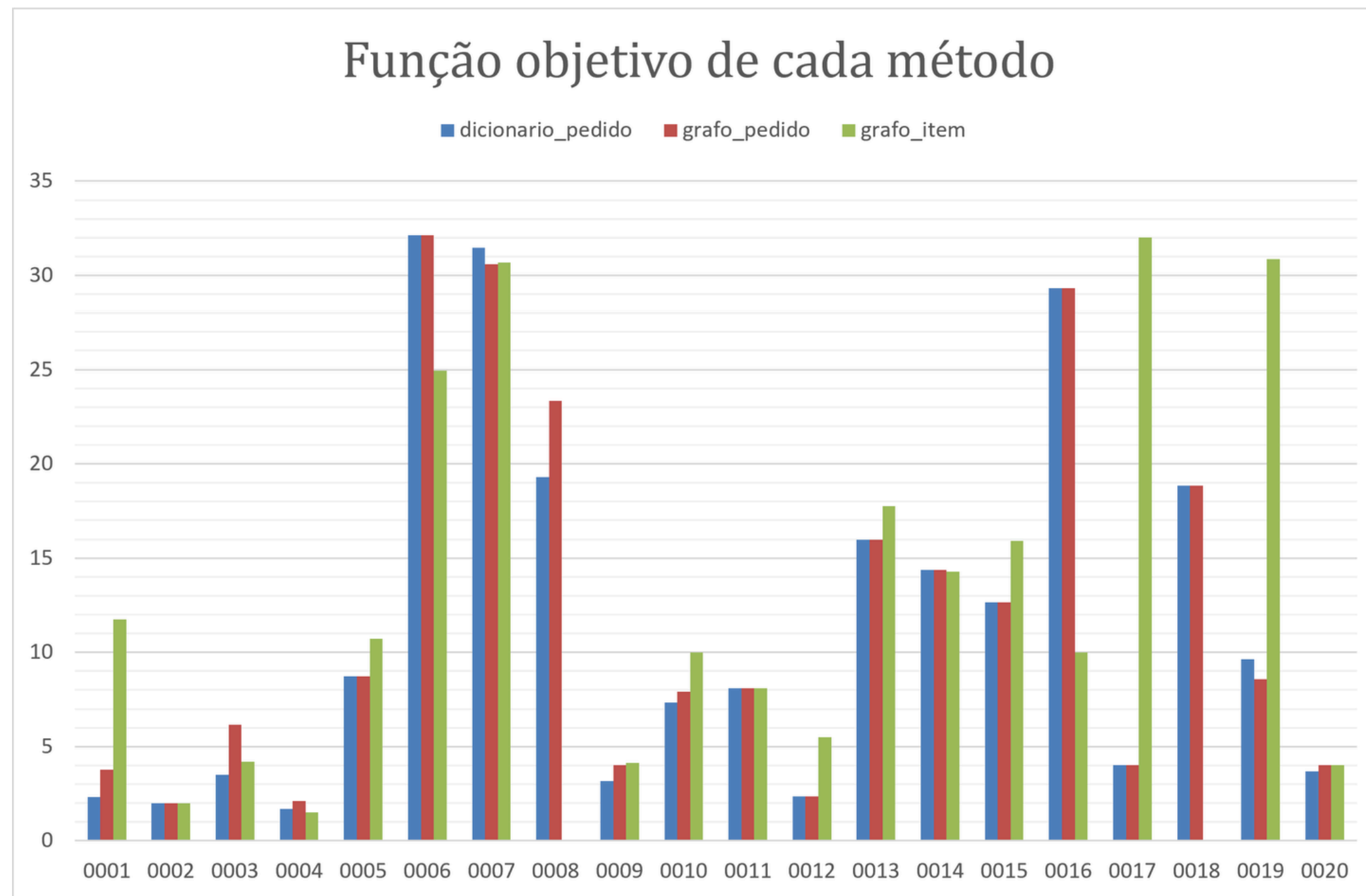


Resultados





Resultados





CONSIDERAÇÕES FINAIS

Obrigado pela atenção!

Dúvidas?